

Módulos

Ingeniería de Sistemas

Universidad Austral — 2026

Tres textos de referencia

Clean Architecture — Robert C. Martin

Mirada centrada en la técnica y en métricas.

Domain-Driven Design — Eric Evans

Mirada centrada en los conceptos del dominio.

On the Criteria To Be Used in Decomposing Systems into Modules — David Parnas
(1972)

El *information hiding* como guía para dividir módulos.

¿Qué es un módulo?

Una **unidad de funcionalidad** en la que descomponemos un programa.

- Expone una **interfaz** para usarlo (API)
- Tiende a ser **reutilizable**
- Permite que un equipo trabaje **en paralelo**: un grupo de personas por módulo

Por qué modularizamos

- Para poder **razonar sobre una parte** sin tener en la cabeza el resto
- Para **absorber cambios** sin que se propaguen por todo el sistema
- Para poder **testear** una parte aislada
- Para que varias personas trabajen **sin pisarse**

Recordar: $n \times (n-1) / 2$ canales de comunicación.

Los módulos son la forma de no pagarlos todos.

Acoplamiento

*Hay acoplamiento si un cambio en uno de mis módulos **requiere** hacer un cambio en otro.*

— Martin Fowler

Bajo acoplamiento hace posible **analizar un módulo con mínimas referencias** a lo que hacen los otros.

Cohesión

*Juntar las cosas que **cambian al mismo tiempo y por las mismas razones**.
Separar las cosas que cambian en momentos distintos y por razones distintas.*

— Robert C. Martin

Cuán relacionadas están las clases dentro de un módulo. La cohesión es lo que hace a un módulo **reusable**.

**Buscamos módulos con
alta cohesión y bajo
acoplamiento**

El criterio de Parnas

La intuición dice: dividir según los **pasos del proceso**.
(*primero esto, después esto otro...*)

Parnas propone otra cosa:

*Cada módulo debe **esconder una decisión de diseño** que es probable que cambie.*

La interfaz es lo que **no** cambia. El resto queda adentro.

Un ejemplo

```
class Authorization {
    boolean canEdit(int userId, int docId) {
        var rs = connection.createStatement()
            .executeQuery("SELECT role FROM users WHERE id = " + userId);
        // ...
    }
}

class User {
    String displayName(int userId) {
        var rs = connection.createStatement()
            .executeQuery("SELECT name, surname FROM users WHERE id = " + userId);
        // ...
    }
}
```

Las preguntas incómodas

- ¿Son **Authorization** y **User** **reutilizables** en otro contexto?
- ¿Qué pasa si **cambia el motor** de base de datos?
- ¿Qué pasa si los datos pasan a estar **en memoria**, o detrás de una **API**?
- ¿Qué pasa si cambia el **esquema** de la tabla?

Cada respuesta implica tocar **los dos** módulos. Y los que vengan después.

Esconder la decisión

```
interface UserRepository {  
    Optional<User> findById(UserId id);  
}
```

Ahora la decisión "los datos están en Postgres con este esquema" vive en **un solo lugar**.

`Authorization` y `User` dependen de una **abstracción**, no de una tabla.

¿Qué detalles escondemos?

- **Cómo** se persiste algo (*motor, esquema, formato*)
- **Cómo** se representa internamente una estructura de datos
- **Cómo** se comunica con un sistema externo (*protocolo, formato, librería*)
- **Cómo** se calcula algo, cuando el algoritmo puede cambiar

Todo lo que empiece con "**cómo**" es candidato. El "**qué**" va en la interfaz.

Principios de cohesión de componentes

REP — *Reuse / Release Equivalence Principle*

La unidad de reutilización es la unidad de *release*.

CCP — *Common Closure Principle*

Juntar las clases que cambian por las mismas razones.

CRP — *Common Reuse Principle*

No obligar a depender de cosas que no se usan.

Principios de acoplamiento entre componentes

ADP — *Acyclic Dependencies Principle*

Sin ciclos en el grafo de dependencias.

SDP — *Stable Dependencies Principle*

Depender siempre en la dirección de la estabilidad.

SAP — *Stable Abstractions Principle*

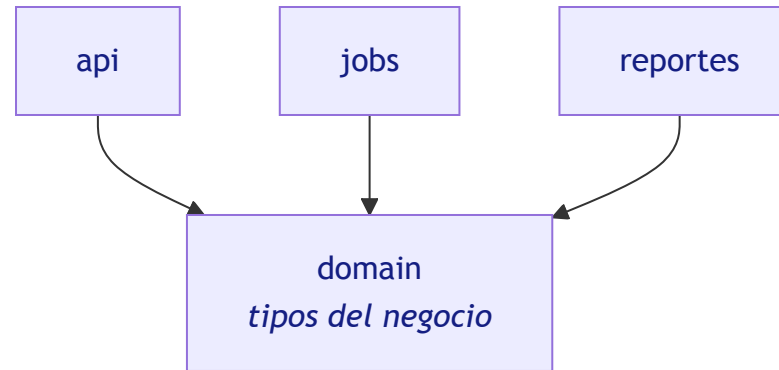
Un componente estable debe ser abstracto.

Stable Dependencies Principle

*Any component that we expect to be **volatile** should not be depended on by a component that is **difficult to change**.*

Otherwise, the volatile component will also be difficult to change.

El módulo más estable



`domain` no depende de nadie y **todos dependen de él**. Eso lo vuelve el módulo más estable — y el más caro de cambiar.

La pregunta que queda

Para cada módulo que diseñen, una sola pregunta:

*¿Qué decisión de diseño estoy escondiendo **acá adentro**?*

¿Preguntas?